

PHASE 2: DEVELOPMENT PHASE

Build a Data Mart in SQL

Airbnb Accommodation Booking Platform

COURSE

DLBDSPBDM01

NAME

Kashish Paresh Gada

MATRICULATION NO.

102202657

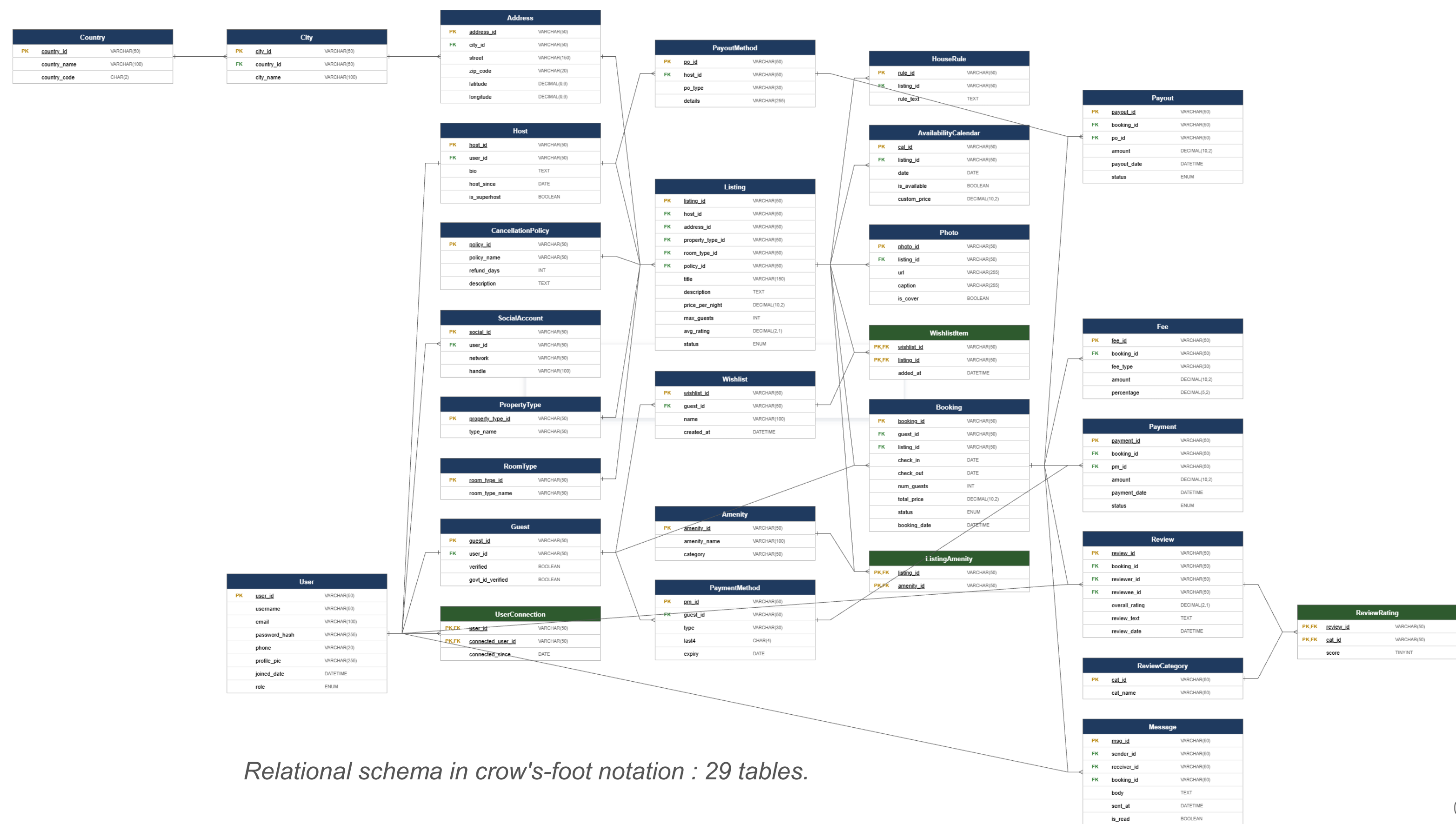
Submission Date

20th July 2026

Project Overview

- Goal: a fully normalised relational database for an Airbnb style accommodation booking platform.
- Models users, listings, bookings, payments, payouts, reviews and messaging.
- 25 entities + 4 junction tables = 29 tables in total.
- Implemented and tested end to end in MySQL 8.

Database Design - ER Model



Relational schema in crow's-foot notation : 29 tables.

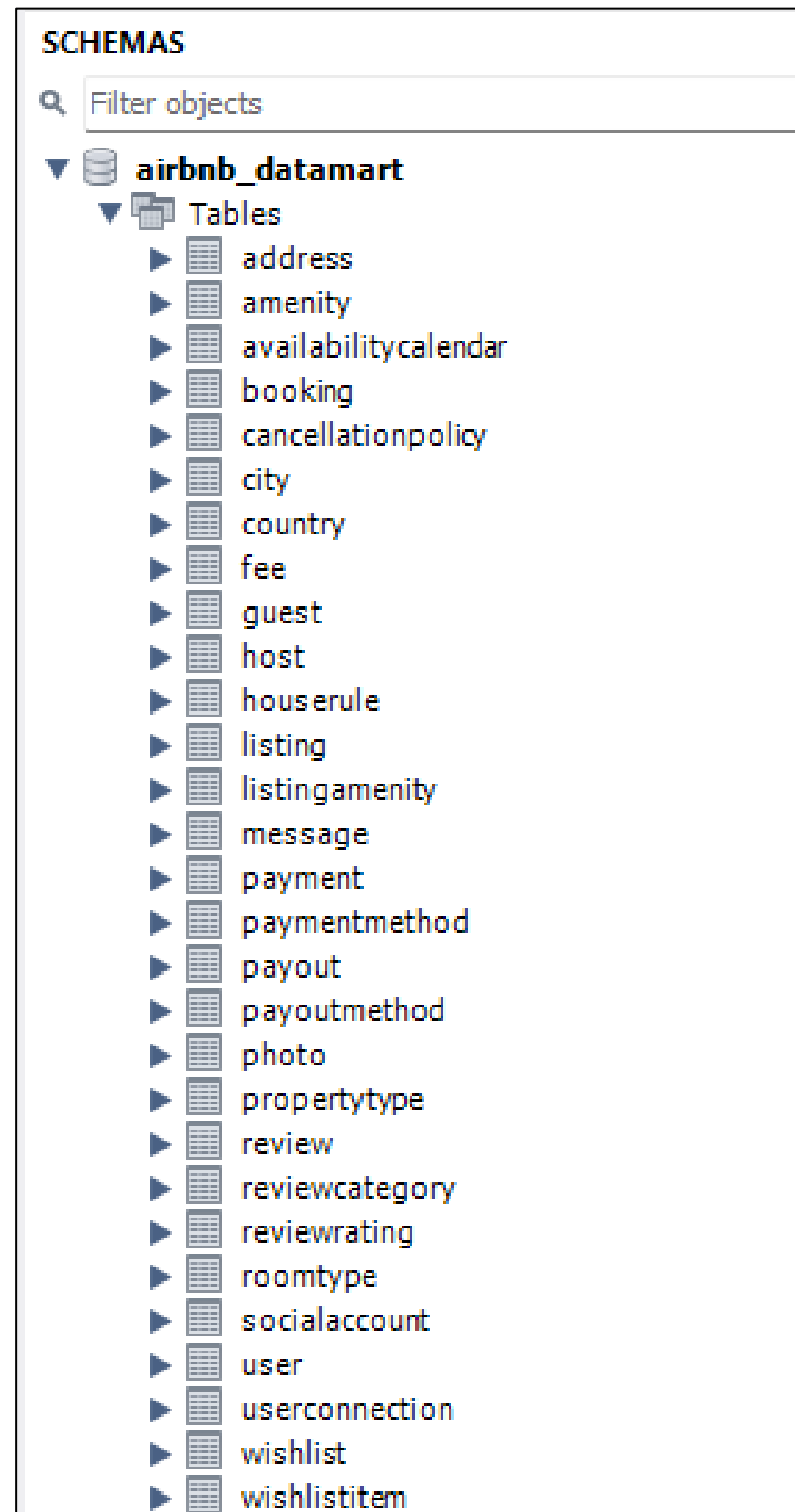
Implementation Approach

- MySQL 8 with the InnoDB storage engine for full referential integrity.
- Readable VARCHAR IDs for clear, human-legible keys.
- Every relationship enforced with a FOREIGN KEY.
- Business rules enforced with CHECK constraints.
- UNIQUE constraints for identity fields and 1:1 relationships.
- Tables created parent-first, so the script runs cleanly top to bottom.

Constraints Used

Constraint	Count	Purpose
PRIMARY KEY	29	One per table.
FOREIGN KEY	37	Every relationship.
UNIQUE	16	Emails, usernames, 1:1 links.
CHECK	19	Ratings 1-5, valid dates, non-negative money, no self-references.

Database Built - Schema



All 29 tables created successfully in MySQL Workbench.

Example Table Definition

```
-- =====
-- 20. BOOKING -- reservation of a listing by a guest
-- =====

• CREATE TABLE `booking` (
  `booking_id`  VARCHAR(50)  NOT NULL,          -- PK
  `guest_id`    VARCHAR(50)  NOT NULL,          -- FK -> guest
  `listing_id`  VARCHAR(50)  NOT NULL,          -- FK -> listing
  `check_in`    DATE         NOT NULL,
  `check_out`   DATE         NOT NULL,
  `num_guests`  INT          NOT NULL DEFAULT 1,
  `total_price` DECIMAL(10,2) NOT NULL,
  `status`      ENUM('PENDING','CONFIRMED','CANCELLED','COMPLETED')
                    NOT NULL DEFAULT 'PENDING',
  `booking_date` DATETIME    NOT NULL DEFAULT CURRENT_TIMESTAMP,

  CONSTRAINT `pk_booking` PRIMARY KEY (`booking_id`),
  CONSTRAINT `fk_booking_guest`
    FOREIGN KEY (`guest_id`) REFERENCES `guest` (`guest_id`)
    ON UPDATE CASCADE ON DELETE RESTRICT,
  CONSTRAINT `fk_booking_listing`
    FOREIGN KEY (`listing_id`) REFERENCES `listing` (`listing_id`)
    ON UPDATE CASCADE ON DELETE RESTRICT,
  CONSTRAINT `chk_booking_dates` CHECK (`check_out` > `check_in`),
  CONSTRAINT `chk_booking_guests` CHECK (`num_guests` > 0),
  CONSTRAINT `chk_booking_total` CHECK (`total_price` >= 0)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

The Booking table showing PK, FKs and CHECK constraints (check_out > check_in).

Test Cases Overview

- 13 SQL test cases validate the design end to end.
- Cover the full business flow: booking → payment → payout → review → messaging.
- Include the ternary and recursive relationships in the model.

End-to-End Booking Scenario

Traces a booking across guest, listing, host, payment and payout (joins 8 tables).

10

-- -----

11

-- TEST CASE 1 (primary): end-to-end booking scenario.

12

-- Traces a booking across guest -> user, listing -> host -> user,

13

-- payment and payout. Proves the core business flow is consistent.

14

-- Joins 8 tables.

15

-- -----

16

• SELECT b.booking_id,

17

gu.username AS guest,

18

l.title AS listing,

19

hu.username AS host,

20

b.check_in, b.check_out, b.total_price,

21

b.status AS booking_status,

22

pay.amount AS amount_paid,

23

pay.status AS payment_status,

24

po.amount AS host_payout,

25

po.status AS payout_status

26

FROM booking b

27

JOIN guest g ON b.guest_id = g.guest_id

28

JOIN `user` gu ON g.user_id = gu.user_id

29

JOIN listing l ON b.listing_id = l.listing_id

30

JOIN host h ON l.host_id = h.host_id

31

JOIN `user` hu ON h.user_id = hu.user_id

32

LEFT JOIN payment pay ON pay.booking_id = b.booking_id

33

LEFT JOIN payout po ON po.booking_id = b.booking_id

34

ORDER BY b.booking_id

35

LIMIT 10;

36

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

Fetch rows:

	booking_id	guest	listing	host	check_in	check_out	total_price	booking_status	amount_paid	payment_status	host_payout	payout_status
►	BKG001	ines.bakker21	Cosy city-centre apartment	liam.jansen1	2026-04-03	2026-04-08	1130.90	COMPLETED	1266.61	PAID	1096.97	RELEASED
	BKG002	marta.weber22	Bright loft with skyline view	emma.muller2	2026-11-20	2026-11-24	876.24	COMPLETED	981.39	PAID	849.95	RELEASED
	BKG003	pavel.bernard23	Charming canal house	noah.dubois3	2026-09-14	2026-09-18	1227.20	COMPLETED	1374.46	PAID	1190.38	RELEASED
	BKG004	sofia.lopez24	Modern studio near park	olivia.garcia4	2026-01-14	2026-01-18	427.12	CONFIRMED	478.37	PAID	414.31	PENDING
	BKG005	hugo.bianchi25	Spacious family villa	ava.rossi5	2026-08-05	2026-08-12	403.97	COMPLETED	452.45	PAID	391.85	RELEASED
	BKG006	elena.costa26	Quiet garden retreat	elijah.silva6	2026-05-20	2026-05-27	1794.31	PENDING	2009.63	FAILED	1740.48	PENDING
	BKG007	anton.brown27	Stylish penthouse suite	sophia.smith7	2026-10-09	2026-10-16	1112.79	CONFIRMED	1246.32	PAID	1079.41	PENDING
	BKG008	klara.kelly28	Rustic countryside cottage	lucas.byrne8	2026-04-15	2026-04-20	1212.75	COMPLETED	1358.28	PAID	1176.37	RELEASED
	BKG009	bjorn.wouters29	Sunny balcony flat	mia.peeters9	2026-03-16	2026-03-20	448.40	COMPLETED	502.21	PAID	434.95	RELEASED
	BKG010	aolife.berger30	Designer duplex downtown	mason.gruber10	2026-05-20	2026-05-24	1161.92	COMPLETED	1301.35	PAID	1127.06	RELEASED

Fee Breakdown

Reconciles the guest service fee (12%) and host service fee (3%) per booking.

37

-- -----

38

-- TEST CASE 2: fee breakdown per booking (guest 12% + host 3%),

39

-- reconciling what the guest paid vs. what the host receives.

40

-- -----

41

•

SELECT b.booking_id, b.total_price,

42

MAX(CASE WHEN f.fee_type='Guest service fee' THEN f.amount END) AS guest_fee,

43

MAX(CASE WHEN f.fee_type='Host service fee' THEN f.amount END) AS host_fee,

44

ROUND(b.total_price + MAX(CASE WHEN f.fee_type='Guest service fee' THEN f.amount END),2) AS guest_pays,

45

ROUND(b.total_price - MAX(CASE WHEN f.fee_type='Host service fee' THEN f.amount END),2) AS host_gets

46

FROM booking b

47

JOIN fee f ON f.booking_id = b.booking_id

48

GROUP BY b.booking_id, b.total_price

49

ORDER BY b.booking_id

50

LIMIT 10;

51

52

-- -----

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

Fetch rows:

	booking_id	total_price	guest_fee	host_fee	guest_pays	host_gets
▶	BKG001	1130.90	135.71	33.93	1266.61	1096.97
	BKG002	876.24	105.15	26.29	981.39	849.95
	BKG003	1227.20	147.26	36.82	1374.46	1190.38
	BKG004	427.12	51.25	12.81	478.37	414.31
	BKG005	403.97	48.48	12.12	452.45	391.85
	BKG006	1794.31	215.32	53.83	2009.63	1740.48
	BKG007	1112.79	133.53	33.38	1246.32	1079.41
	BKG008	1212.75	145.53	36.38	1358.28	1176.37
	BKG009	448.40	53.81	13.45	502.21	434.95
	BKG010	1161.92	139.43	34.86	1301.35	1127.06

Revenue Per Host

Total released payouts by host.

52

-- -----

53

-- TEST CASE 3: revenue per host (released payouts only).

54

-- -----

55

• SELECT hu.username AS host,

56

COUNT(DISTINCT b.booking_id) AS bookings,

57

ROUND(SUM(CASE WHEN po.status='RELEASED' THEN po.amount ELSE 0 END),2) AS released_payout

58

FROM host h

59

JOIN `user` hu ON h.user_id = hu.user_id

60

JOIN listing l ON l.host_id = h.host_id

61

JOIN booking b ON b.listing_id = l.listing_id

62

JOIN payout po ON po.booking_id= b.booking_id

63

GROUP BY h.host_id, hu.username

64

ORDER BY released_payout DESC

65

LIMIT 10;

66

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

Fetch rows:

	host	bookings	released_payout
▶	liam.jansen1	3	2411.11
	harper.novak17	1	1971.20
	jonas.papadopoulos18	1	1328.53
	noah.dubois3	1	1190.38
	lucas.byrne8	1	1176.37
	mason.gruber10	1	1127.06
	amelia.hansen13	1	865.14
	emma.muller2	1	849.95
	charlotte.korhonen15	1	770.96
	nora.horvat19	1	726.07

Bookings & Revenue by Geography

Bookings and revenue by country and city.

67

-- -----

68

-- TEST CASE 4: bookings and revenue by country and city

69

-- (booking -> listing -> address -> city -> country).

70

-- -----

71

•

SELECT co.country_name, ci.city_name,

72

COUNT(b.booking_id) AS bookings,

73

ROUND(SUM(b.total_price),2) AS revenue

74

FROM booking b

75

JOIN listing l ON b.listing_id = l.listing_id

76

JOIN address a ON l.address_id = a.address_id

77

JOIN city ci ON a.city_id = ci.city_id

78

JOIN country co ON ci.country_id = co.country_id

79

GROUP BY co.country_id, ci.city_id

80

ORDER BY revenue DESC

81

LIMIT 10;

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

Fetch rows:

	country_name	city_name	bookings	revenue
▶	United States	New York	2	2322.48
	Austria	Vienna	1	2032.17
	France	Lyon	1	1794.31
	Switzerland	Zurich	1	1369.62
	United Kingdom	Manchester	1	1326.95
	Germany	Berlin	1	1227.20
	Spain	Barcelona	1	1212.75
	Italy	Milan	1	1161.92
	Netherlands	Amsterdam	1	1130.90
	Spain	Madrid	1	1112.79

Top Guests by Amount Paid

Top guests by total amount paid.

83

-- -----

84

-- TEST CASE 5: top guests by total amount actually paid.

85

-- -----

86

• SELECT gu.username AS guest,

87

COUNT(p.payment_id) AS payments,

88

ROUND(SUM(p.amount),2) AS total_paid

89

FROM payment p

90

JOIN booking b ON p.booking_id = b.booking_id

91

JOIN guest g ON b.guest_id = g.guest_id

92

JOIN `user` gu ON g.user_id = gu.user_id

93

WHERE p.status = 'PAID'

94

GROUP BY g.guest_id, gu.username

95

ORDER BY total_paid DESC

96

LIMIT 10;

97

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

Fetch rows:

	guest	payments	total_paid
▶	ines.bakker21	2	2783.96
	timo.nikolaou37	1	2276.03
	alma.maric38	1	1533.97
	pavel.bernard23	1	1374.46
	klara.kelly28	1	1358.28
	aoife.berger30	1	1301.35
	anton.brown27	1	1246.32
	tom.ricci43	1	1083.82
	milan.larsen33	1	998.93
	marta.weber22	1	981.39

Ternary Relationship - Reviews

A review linking a booking to a reviewer and a reviewee.

```
98  -- -----
99  -- TEST CASE 6: TERNARY relationship - Review links a Booking to a
100 -- reviewer (User) and a reviewee (User). Joins review + booking + user twice.
101  -- -----
102 • SELECT r.review_id, b.booking_id,
103        ru.username AS reviewer, re.username AS reviewee,
104        r.overall_rating, r.review_text
105 FROM review r
106 JOIN booking b ON r.booking_id = b.booking_id
107 JOIN `user` ru ON r.reviewer_id = ru.user_id
108 JOIN `user` re ON r.reviewee_id = re.user_id
109 ORDER BY r.review_id
110 LIMIT 10;
111
112  -- -----
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

	review_id	booking_id	reviewer	reviewee	overall_rating	review_text
▶	REV001	BKG001	ines.bakker21	liam.jansen1	4.5	Host was responsive and helpful.
	REV002	BKG002	marta.weber22	emma.muller2	4.5	Exactly as described, lovely place.
	REV003	BKG003	pavel.bernard23	noah.dubois3	3.5	Host was responsive and helpful.
	REV004	BKG005	hugo.bianchi25	ava.rossi5	5.0	Comfortable and quiet, would book again.
	REV005	BKG008	klara.kelly28	lucas.byrne8	4.5	Exactly as described, lovely place.
	REV006	BKG009	bjorn.wouters29	mia.peeters9	3.5	Exactly as described, lovely place.
	REV007	BKG010	aoife.berger30	mason.gruber10	4.5	Smooth check-in, great value.
	REV008	BKG011	sven.karlsson31	isabella.meier11	3.5	Host was responsive and helpful.
	REV009	BKG012	lea.olsen32	ethan.andersson12	5.0	Comfortable and quiet, would book again.
	REV010	BKG013	milan.larsen33	amelia.hansen13	3.5	Smooth check-in, great value.

Average Score per Review Category

Average score per review category.

112
113
114
115
116
117
118
119
120
121
122
123
124

```
-- -----  
-- TEST CASE 7: average score per review category (review -> reviewrating  
-- -> reviewcategory).  
-- -----  
SELECT rc.cat_name,  
       ROUND(AVG(rr.score),2) AS avg_score,  
       COUNT(*) AS num_ratings  
FROM reviewrating rr  
JOIN reviewcategory rc ON rr.cat_id = rc.cat_id  
GROUP BY rc.cat_id, rc.cat_name  
ORDER BY avg_score DESC  
LIMIT 10;
```

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

Fetch rows:

	cat_name	avg_score	num_ratings
▶	Check-in	4.67	3
	Accuracy	4.60	5
	Wi-Fi quality	4.60	5
	Cleanliness	4.50	4
	Overall experience	4.33	6
	Kitchen quality	4.25	4
	Hospitality	4.17	6
	Comfort	4.14	7
	Safety	4.13	8
	Ambience	4.00	2

Average Guest Rating per Listing

Average guest rating per listing.

125
126
127
128 •
129
130
131
132
133
134
135
136
137

```
-- -----  
-- TEST CASE 8: average guest rating per listing (from reviews).  
-- -----  
SELECT l.title,  
       ROUND(AVG(r.overall_rating),2) AS avg_rating,  
       COUNT(r.review_id) AS reviews  
FROM review r  
JOIN booking b ON r.booking_id = b.booking_id  
JOIN listing l ON b.listing_id = l.listing_id  
GROUP BY l.listing_id, l.title  
ORDER BY avg_rating DESC  
LIMIT 10;
```

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

Fetch rows:

	title	avg_rating	reviews
▶	Romantic attic nest	5.00	1
	Historic townhouse gem	5.00	2
	Cosy city-centre apartment	4.75	2
	Bright loft with skyline view	4.75	2
	Spacious family villa	4.75	2
	Rustic countryside cottage	4.75	2
	Green rooftop hideaway	4.75	2
	Designer duplex downtown	4.50	2
	Luxury chalet with hot tub	4.50	1
	Boutique guest suite	4.50	2

Most-Wishlisted Listings

Most-wishlisted listings (many-to-many).

138

-- -----

139

-- TEST CASE 9: most-wishlisted listings (M:N wishlist <-> listing).

140

-- -----

141

•

SELECT l.title, COUNT(wi.wishlist_id) AS times_wishlisted

142

FROM wishlistitem wi

143

JOIN listing l ON wi.listing_id = l.listing_id

144

GROUP BY l.listing_id, l.title

145

ORDER BY times_wishlisted DESC

146

LIMIT 10;

147

148

-- -----

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

Fetch rows:

	title	times_wishlisted
▶	Cosy city-centre apartment	2
	Bright loft with skyline view	2
	Charming canal house	2
	Modern studio near park	2
	Spacious family villa	2
	Quiet garden retreat	2
	Stylish penthouse suite	2
	Rustic countryside cottage	2
	Sunny balcony flat	2
	Designer duplex downtown	2

Amenity Popularity

Amenity popularity (many-to-many).

148

-- -----

149

-- TEST CASE 10: amenity popularity (M:N listing <-> amenity).

150

-- -----

151

•

SELECT am.amenity_name, COUNT(la.listing_id) AS listings_offering

152

FROM listingamenity la

153

JOIN amenity am ON la.amenity_id = am.amenity_id

154

GROUP BY am.amenity_id, am.amenity_name

155

ORDER BY listings_offering DESC

156

LIMIT 10;

157

158

-- -----

Result Grid

Filter Rows:

Exports

Wrap Cell Content:

Fetch rows:

	amenity_name	listings_offering
▶	Air conditioning	3
	Balcony	3
	Hair dryer	3
	Coffee maker	3
	Wi-Fi	3
	Dishwasher	3
	Gym	3
	Elevator	3
	Fire extinguisher	3
	First aid kit	3

Recursive Relationship - Connections

A user's social connections.

158

-- -----

159

-- TEST CASE 11: RECURSIVE relationship - a user's social connections

160

-- (userconnection joins user to user).

161

-- -----

162

•

SELECT u1.username AS user, u2.username AS connected_to, uc.connected_since

163

FROM userconnection uc

164

JOIN `user` u1 ON uc.user_id = u1.user_id

165

JOIN `user` u2 ON uc.connected_user_id = u2.user_id

166

ORDER BY u1.username

167

LIMIT 10;

168

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

Fetch rows:

	user	connected_to	connected_since
▶	alma.maric38	lea.olsen32	2025-09-22
	anton.brown27	nora.horvat19	2025-02-13
	ava.rossi5	mason.gruber10	2025-12-21
	ava.rossi5	ines.bakker21	2025-10-14
	charlotte.korhonen15	milan.larsen33	2025-09-27
	elijah.silva6	elena.costa26	2025-05-24
	elijah.silva6	klara.kelly28	2025-02-08
	emma.muller2	elijah.silva6	2025-12-17
	emma.muller2	ines.bakker21	2025-03-26
	ethan.andersson12	mia.peeters9	2025-07-18

Users as Both Host & Guest

Users who act as both host and guest.

169

-- -----

170

-- TEST CASE 12: users who act as BOTH host and guest

171

-- (a user having a row in host AND in guest).

172

-- -----

173

•

SELECT u.user_id, u.username, u.role

174

FROM `user` u

175

JOIN host h ON h.user_id = u.user_id

176

JOIN guest g ON g.user_id = u.user_id

177

ORDER BY u.user_id;

178

179

-- -----

Result Grid

 Filter Rows:

Export: 

Wrap Cell Content: 

	user_id	username	role
▶	USR041	nikola.fischer41	BOTH
	USR042	sara.moreau42	BOTH
	USR043	tom.ricci43	BOTH

Payment Status Summary

Payment status summary (data-quality overview).

179

-- -----

180

-- TEST CASE 13: payment status summary (data-quality overview).

181

-- -----

182

•

SELECT status, COUNT(*) AS num_payments, ROUND(SUM(amount),2) AS total_amount

183

FROM payment

184

GROUP BY status

185

ORDER BY num_payments DESC;

186

187

-- End of queries.sql

188

189

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

	status	num_payments	total_amount
▶	PAID	21	20319.03
	REFUNDED	2	2332.03
	FAILED	1	2009.63

Summary & Conclusion

- 29-table normalised database fully implemented in MySQL.
- All relationships and business rules enforced with constraints.
- 20+ rows per table forming complete, consistent scenarios.
- 13 test cases confirm the design works end to end.
- Ready for the finalisation phase.